

uc3m

Universidad
Carlos III
de Madrid



Práctica 1: Parser descendiente recursivo

Procesadores del lenguaje

**Manolo Pedro
Pitudo pitudon**

10 de Marzo de 2026

Índice

1. Introducción	3
2. ¿Qué elementos léxicos proporciona la función <code>rd_lex()</code> ?	3
3. Construcción de la gramática	5

Introducción

En esta primera práctica se lleva a cabo el desarrollo de un traductor de notación prefija a infija en operaciones básicas de una calculadora. Para ello empleamos un Analizador Descendente Recursivo implementado en C en el archivo drLL.c a partir de una gramática cuyo funcionamiento y desarrollo también se abarcan en esta práctica

¿Qué elementos léxicos proporciona la función `rd_lex()`?

Primeramente analizamos el fichero drLL.c proporcionado, con el analizador `rd_lex()` ya programado y sin necesidad de realizar cambios.

En esta parte del código previa a la función (al inicio del fichero):

```
struct s_tokens {
    int token ;           // Here we store the current token/literal
    int old_token ;       // Sometimes we need to check the previous
token
    int number ;          // The value of the number
    int old_number ;      // old number value
    char variable_name [8] ;    /// variable name
    char old_var_name [8] ;     /// old variable name
    int token_val ;        // the arithmetic operator
    int old_token_val ;     // old arithmetic operator
} ;
```

Vemos que el tipo de dato que se usa para almacenar los tokens es un struct que tiene el doble de campos que tokens admitidos para poder almacenar cualquier tipo de token y también el anterior en caso necesario.

Mirando esta sección del código ya podemos intuir que los tokens que se van a devolver son:

1. Número (`number`)
2. Nombre de variable (`variable_name`)
3. Operador (`token_val`)

Inspeccionando la función `rd_lex` en sí misma encontramos que:

1. En la primera parte de la función:

```
do {
    c = getchar () ;
    if (c == '\n')
        line_counter++ ; // info for rd_syntax_error()
} while (c == '\t' || c == ' ' || c == '\r') ;
```

Vemos que se ignoran los caracteres tab («\t»), espacio (« ») y el caracter de retorno («\r») ya que si el programa encuentra cualquiera de ellos sigue extrayendo caracteres del flujo de entrada.

2. Examinando el primer condicional:

```

if (isdigit (c)) {      /// Token Number is [Digit]+
    ungetc (c, stdin) ;    /// This returns one character to the
    standard input stream
    update_old_token () ;
    scanf ("%d", &tokens.number) ;
    tokens.token = T_NUMBER ;
    return (tokens.token) ; // returns the Token for Variable
}

```

Podemos ver que se intenta capturar un número, siendo este el primer tipo de token que nos puede devolver la función `rd_lex()`. Esto lo hace actualizando el token anterior (por si nos hiciera falta),

```
update_old_token()
```

devolviendo el caracter leído al flujo de entrada:

```
ungetc(c, stdin)
```

y leyendo directamente un número completo del flujo de entrada, introduciendolo en el atributo *number* del struct *tokens*.

```
update_old_token()
```

Igualmente guarda un número preestablecido en el atributo *number* del mismo struct para indicar que el último valor guardado es un número.

```
tokens.token = T_NUMBER
```

Habiendo definido previamente `T_NUMBER` como una constante a través de un `define` como se hace con los demás posibles números identificativos de tokens:

```

#define T_NUMBER 1001
#define T_OPERATOR 1002
#define T_VARIABLE 1003

```

Finalmente devuelve el código del token:

```
return (tokens.token)
```

3. En el siguiente condicional:

```

if (isalpha(c)) { /// Token Variable of type Letter[Digit|Letter]?
    update_old_token () ;
    cc = getchar () ;
    if (isdigit (cc) || isalpha (cc)) {
        sprintf (tokens.variable_name, "%c%c", c, cc) ;    /// This
copies the Letter.Digit|Letter name in the variable name
    } else {
        ungetc (cc, stdin) ;
        sprintf (tokens.variable_name, "%c", c) ;    /// This copies
the single Letter name in the variable name
    }
    tokens.token = T_VARIABLE ;
    return (tokens.token) ; // returns the Token for Variable
}

```

Se realiza una lectura del token *VARIABLE* y funciona de manera análoga al anterior, con una única diferencia que recae en la extracción de los caracteres que componen el token, ya que, en este segundo condicional, el siguiente caracter que compone el token se extrae manualmente en vez de usar `printf`, usando:

```
cc = getchar ()
```

De esta manera se comprueba si la variable es de la forma Letra o Letra + Letra | Dígito

4. En el último condicional:

```
if (c == '+' || c == '-' || c == '*' || c == '/') { // Remember
that OTHER SYMBOLS ARE returned as literals
    update_old_token ();
    tokens.token_val = c ;
    tokens.token = T_OPERATOR ;
    return (tokens.token) ; // returns the Token for Arithmetic
Operators
}
```

Se extrae el token *OPERADOR*, compuesto por un sólo caracter que puede ser +, -, * o /.

Entonces, tras analizar la función `rd_lex()` podemos concluir que efectivamente se devuelven 3 tokens:

- *Operador*
- *Variable*
- *Número*

Construcción de la gramática

Para empezar a desarrollar la gramática comenzaremos con una gramática simple que nos permita reconocer las operaciones básicas tal como +, -, /, *; centrándonos exclusivamente en la funcionalidad correspondiente a los calculos, dejando la inclusión de variables para más tarde:

```
S ::= En
E ::= 0EE | (E) | N
0 ::= + | - | * | /
N ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

No Terminales: S: Símbolo inicial E: Expresión 0: Operador N: Dígito numérico

Terminales: n: Salto de línea (): Paréntesis + - * /: Operaciones 0...9: Cifras

Listado 1: Gramática de expresiones prefijas con soporte de paréntesis anidados

Esta gramática inicial tiene un pequeño problema, que permite la creación de construcciones del tipo $((((E))))$, claramente prohibidas por el enunciado, por lo que la modificaremos con la introducción de un nuevo no terminal D que nos permita construir sólo una expresión con paréntesis, modificando a su vez el no terminal E para que no permita introducir paréntesis.

$$\begin{aligned}
S &::= Dn \\
D &::= E \mid (E) \\
E &::= OEE \mid N \\
O &::= + \mid - \mid * \mid / \\
N &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{aligned}$$

No Terminales: S: Símbolo inicial D: Derivación intermedia para evitar paréntesis anidados E: Expresión O: Operador aritmético N: Dígito numérico

Terminales: n: Fin de línea (): Paréntesis + - * /: Operadores 0...9: Cifras decimales

Listado 2: Gramática inicial simple para el parser descendente

Por suerte, esta nueva gramática, ya es LL(1) ya que se cumplen todas las condiciones:

- No hay recursividad por la derecha ni directa ni indirecta.
- No tenemos que realizar ninguna factorización por la izquierda ya que no tenemos conflictos primero-primero
- Al no tener cadenas vacías, tampoco tenemos condiciones primero-siguiente

Ya que disponemos de esta gramática simple que por el momento es correcta, podemos continuar complicandola para que represente todas las posibles construcciones: Para ello, primero extendemos los números para que puedan tener múltiples dígitos. Introducimos un no terminal Num que genera una secuencia de uno o más dígitos, utilizando recursividad por la derecha y factorizando con RestoNum para mantener la gramática LL(1):

$$\begin{aligned}
S &::= Dn \\
D &::= E \mid (E) \\
E &::= OEE \mid \text{Num} \\
O &::= + \mid - \mid * \mid / \\
\text{Num} &::= \text{dígito RestoNum} \\
\text{RestoNum} &::= \text{dígito RestoNum} \mid \lambda
\end{aligned}$$

No Terminales: S: Símbolo inicial D: Derivación intermedia E: Expresión O: Operador aritmético Num: Número de uno o más dígitos RestoNum: Parte recursiva para números

Terminales: n: Fin de línea (): Paréntesis
+ - * /: Operadores
dígito: Cualquier dígito del 0 al 9

Figura 1: Gramática con números de múltiples dígitos

A continuación, añadimos las variables. Una variable puede ser una letra seguida opcionalmente de más letras o dígitos. Para ello definimos un nuevo terminal letra que representa cualquier carácter alfabético, y creamos producciones recursivas por la derecha:

```

S ::= D n
D ::= E | (E)
E ::= OEE | Num | Var
O ::= 1. | - | * | /
Num ::= digito RestoNum
RestoNum ::= digito RestoNum | λ
Var ::= letra RestoVar
RestoVar ::= letra RestoVar | digito RestoVar | λ

```

No Terminales: S: Símbolo inicial D: Derivación intermedia E: Expresión O: Operador aritmético Num: Número RestoNum: Parte recursiva para números Var: Variable RestoVar: Parte recursiva para variables

Terminales: n: Fin de línea (): Paréntesis
1. • */: Operadores
digito: Dígito 0-9 letra: Letra mayúscula o minúscula a-z, A-Z

Figura 2: Gramática con variables

Ahora incorporamos los operadores de asignación (=) y ternario (?). Estos operadores, al igual que los aritméticos, se aplican a expresiones y deben ir entre paréntesis. Para mantener la gramática LL(1) y evitar ambigüedades, creamos un nuevo no terminal OpApp que agrupa todas las posibles aplicaciones de operadores (aritméticos, asignación y ternario). La expresión E puede ser un número, una variable o una aplicación de operador entre paréntesis. Así, la gramática final queda:

```

S ::= E n
E ::= Num | Var | ( OpApp )
OpApp ::= O E E
        | "=" Var E
        | "?" E E E
O ::= "+" | "-" | "*" | "/"
Num ::= digito RestoNum
RestoNum ::= digito RestoNum | λ
Var ::= letra RestoVar
RestoVar ::= letra RestoVar | digito RestoVar | λ

```

No Terminales: S: Símbolo inicial E: Expresión OpApp: Aplicación de operador O: Operador aritmético Num: Número RestoNum: Parte recursiva para números Var: Variable RestoVar: Parte recursiva para variables

Terminales: n: Fin de línea (): Paréntesis
1. • */: Operadores aritméticos
=: Operador de asignación ?: Operador ternario digito: Dígito 0-9 letra: Letra mayúscula o minúscula

Figura 3: Gramática final con asignación y operador ternario

Esta gramática ya incluye todas las construcciones requeridas: números de múltiples dígitos, variables, operaciones aritméticas, asignaciones y el operador ternario. Además, al haber restringido el uso de paréntesis únicamente alrededor de OpApp, se evitan los paréntesis anidados sin operador (como ((E))), cumpliendo así las restricciones del enunciado. Para verificar que es LL(1), calculamos los conjuntos Primero y Siguiente. Los

no terminales anulables son RestoNum y RestoVar. No existen conflictos primero-primero ni primero-siguiente, por lo que la gramática es LL(1) y podemos implementar un parser descendente recursivo basado en ella.